

Lab 3 – Construction of Token Generator

Q.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#include <stdlib.h>
FILE *source_file;
int line_num = 1, column = 0;
char *token_list[] = {"int", "main", "if", "else", "while", "return", "char",
"float", "void", "for"};
int kw_size = 10;
char peek_char = '\0';
int has_peek = 0;

int checkKeyword(char *str) {
    for (int i = 0; i < kw_size; i++)
        if (strcmp(str, token_list[i]) == 0) return 1;
    return 0;
}

int checkArithmetic(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '%' || c == '/');
}

int checkRelational(char c) {
    return (c == '<' || c == '>' || c == '!' || c == '=');
}

int checkSpecial(char c) {
    return (c == '(' || c == ')' || c == '{' || c == '}' || c == '[' || c == ']'
|| c == ';' || c == ',');
}

char nextChar() {
    if (has_peek) {
        has_peek = 0;
        return peek_char;
    }
    char c = fgetc(source_file);
    if (c != EOF) column++;
    return c;
}
```

```

}
void retract(char c) {
    peek_char = c;
    has_peek = 1;
    if (c != EOF) column--;
}
void skipDirective() {
    char tag[10], ch;
    int idx = 0;
    while ((ch = nextChar()) == ' ');
    while (isalpha(ch) && idx < 9) {
        tag[idx++] = ch;
        ch = nextChar();
    }
    tag[idx] = '\0';
    if (!strcmp(tag, "include") || !strcmp(tag, "define")) {
        while (ch != '\n' && ch != EOF) ch = nextChar();
        line_num++;
        column = 0;
    } else {
        retract(ch);
    }
}
void skipComments(char type) {
    char ch;
    if (type == '/') {
        while ((ch = nextChar()) != '\n' && ch != EOF);
        line_num++; column = 0;
    } else {
        while (1) {
            ch = nextChar();
            if (ch == EOF) break;
            if (ch == '\n') { line_num++;
                column = 0;
            }
            if (ch == '*') {
                char next = nextChar();
                if (next == '/') break;
                else retract(next);
            }
        }
    }
}
int findNextToken() {

```

```

char ch, lexeme[100];
int j;
while ((ch = nextChar()) != EOF) {
    if (ch == ' ' || ch == '\t') continue;
    if (ch == '\n') { line_num++;
        column = 0;
        continue;
    }
    if (ch == '#') {
        skipDirective();
        continue;
    }
    if (ch == '/') {
        char next = nextChar();
        if (next == '/' || next == '*') {
            skipComments(next);
            continue;
        }
        retract(next);
    }
    if (ch == '"') {
        j = 0; lexeme[j++] = ch;
        while ((ch = nextChar()) != '"' && ch != EOF) {
            if (ch == '\n') {
                line_num++;
                column = 0;
            }
            lexeme[j++] = ch;
        }
        lexeme[j++] = '"'; lexeme[j] = '\0';
        printf("<%s , %d , %d , STRING>\n", lexeme, line_num, column -
(int)strlen(lexeme) + 1);
        return 1;
    }
    if (isalpha(ch) || ch == '_') {
        j = 0; lexeme[j++] = ch;
        while (isalnum(ch = nextChar()) || ch == '_') lexeme[j++] = ch;
        lexeme[j] = '\0';
        retract(ch);
        if (checkKeyword(lexeme))
            printf("<%s , %d , %d , KEYWORD>\n", lexeme, line_num, column -
(int)strlen(lexeme) + 1);
        else

```

```

        printf("<%s , %d , %d , IDENTIFIER>\n", lexeme, line_num, column
- (int)strlen(lexeme) + 1);
        return 1;
    }
    if (isdigit(ch)) {
        j = 0; lexeme[j++] = ch;
        while (isdigit(ch = nextChar())) lexeme[j++] = ch;
        lexeme[j] = '\0'; retract(ch);
        printf("<%s , %d , %d , NUMBER>\n", lexeme, line_num, column -
(int)strlen(lexeme) + 1);
        return 1;
    }
    if (checkArithmetic(ch)) {
        printf("<%c , %d , %d , ARITHMETIC>\n", ch, line_num, column);
        return 1;
    }
    if (checkRelational(ch)) {
        char op[3] = {ch, '\0', '\0'};
        char next = nextChar();
        if (next == '=') {
            op[1] = '=';
            printf("<%s , %d , %d , RELOP>\n", op, line_num, column - 1);
        } else {
            retract(next);
            if (ch == '=') printf("<%s , %d , %d , ASSIGNMENT>\n", op,
line_num, column);
            else printf("<%s , %d , %d , RELOP>\n", op, line_num, column);
        }
        return 1;
    }
    if (ch == '&' || ch == '|') {
        char next = nextChar();
        if (next == ch) {
            printf("<%c%c , %d , %d , LOGIC>\n", ch, next, line_num, column -
1);
            return 1;
        }
        retract(next);
    }
    if (checkSpecial(ch)) {
        printf("<%c , %d , %d , SPECIAL SYMBOL>\n", ch, line_num, column);
        return 1;
    }
}

```

```

        return 0;
    }
int main() {
    source_file = fopen("input.c", "r");
    if (!source_file) {
        printf("Error: Cannot open source file.\n");
        return 0;
    }
    while (findNextToken());
    fclose(source_file);
    return 0;
}

```

Input.c

```

#include <stdio.h>
#define PI 3.14

/* This is a multi-line
   comment to test the lexer
*/

int main() {
    int radius = 10; // This is a single-line comment
    char msg[] = "Lexical analysis test";
    float area;

    if (radius >= 10 && radius != 0) {
        area = PI * radius * radius;
        printf("Area is: %f", area);
    } else {
        return 0;
    }

    /* Testing logic and arithmetic */
    int x = (5 + 3) * 2;

    return 0;
}

```

Output:

```
PS C:\_Work_\RANDOM\CD_Hiten_L3> gcc -o abc q.c
```

```
PS C:\_Work_\RANDOM\CD_Hiten_L3> ./abc
```

```
<int , 8 , 1 , KEYWORD>
```

```
<main , 8 , 4 , KEYWORD>
```

```
<( , 8 , 7 , SPECIAL SYMBOL>
```

```
< ) , 8 , 8 , SPECIAL SYMBOL>
```

```
<{ , 8 , 10 , SPECIAL SYMBOL>
```

```
<int , 9 , 5 , KEYWORD>
```

```
<radius , 9 , 8 , IDENTIFIER>
```

```
<= , 9 , 14 , ASSIGNMENT>
```

```
<10 , 9 , 15 , NUMBER>
```

```
<; , 9 , 16 , SPECIAL SYMBOL>
```

```
<char , 10 , 5 , KEYWORD>
```

```
<msg , 10 , 9 , IDENTIFIER>
```

```
<[ , 10 , 11 , SPECIAL SYMBOL>
```

```
<] , 10 , 12 , SPECIAL SYMBOL>
```

```
<= , 10 , 14 , ASSIGNMENT>
```

```
<"Lexical analysis test" , 10 , 15 , STRING>
```

```
<; , 10 , 38 , SPECIAL SYMBOL>
```

```
<float , 11 , 5 , KEYWORD>
```

```
<area , 11 , 10 , IDENTIFIER>
```

```
<; , 11 , 13 , SPECIAL SYMBOL>
```

```
<if , 13 , 5 , KEYWORD>
```

```
<( , 13 , 7 , SPECIAL SYMBOL>
```

```
<radius , 13 , 8 , IDENTIFIER>
```

```
<>= , 13 , 14 , RELOP>
```

```
<10 , 13 , 17 , NUMBER>
```

```
<&& , 13 , 19 , LOGIC>
```

```
<radius , 13 , 22 , IDENTIFIER>
```

```
<!= , 13 , 28 , RELOP>
<0 , 13 , 31 , NUMBER>
<)> , 13 , 31 , SPECIAL SYMBOL>
<{ , 13 , 33 , SPECIAL SYMBOL>
<area , 14 , 9 , IDENTIFIER>
<= , 14 , 13 , ASSIGNMENT>
<PI , 14 , 14 , IDENTIFIER>
<* , 14 , 16 , ARITHMETIC>
<radius , 14 , 18 , IDENTIFIER>
<* , 14 , 24 , ARITHMETIC>
<radius , 14 , 26 , IDENTIFIER>
<; , 14 , 31 , SPECIAL SYMBOL>
<printf , 15 , 9 , IDENTIFIER>
<( , 15 , 14 , SPECIAL SYMBOL>
<"Area is: %f" , 15 , 15 , STRING>
< , 15 , 28 , SPECIAL SYMBOL>
<area , 15 , 30 , IDENTIFIER>
<)> , 15 , 33 , SPECIAL SYMBOL>
<; , 15 , 34 , SPECIAL SYMBOL>
<} , 16 , 5 , SPECIAL SYMBOL>
<else , 16 , 7 , KEYWORD>
<{ , 16 , 11 , SPECIAL SYMBOL>
<return , 17 , 9 , KEYWORD>
<0 , 17 , 15 , NUMBER>
<; , 17 , 15 , SPECIAL SYMBOL>
<} , 18 , 5 , SPECIAL SYMBOL>
<int , 21 , 5 , KEYWORD>
```

```
<x , 21 , 8 , IDENTIFIER>  
<= , 21 , 9 , ASSIGNMENT>  
<( , 21 , 10 , SPECIAL SYMBOL>  
<5 , 21 , 11 , NUMBER>  
<+ , 21 , 12 , ARITHMETIC>  
<3 , 21 , 14 , NUMBER>  
< ) , 21 , 14 , SPECIAL SYMBOL>  
<* , 21 , 16 , ARITHMETIC>  
<2 , 21 , 18 , NUMBER>  
<; , 21 , 18 , SPECIAL SYMBOL>  
<return , 23 , 5 , KEYWORD>  
<0 , 23 , 11 , NUMBER>  
<; , 23 , 11 , SPECIAL SYMBOL>  
<} , 24 , 1 , SPECIAL SYMBOL>  
PS C:\ Work \RANDOM\CD Hiten L3>
```